

PORTFOLIO

이름	박준수
이메일	pjsu94@naver.com
GitHub	github.com/kdm111

PROJECTS



개인 프로젝트

[프로젝트 코드 ↗](#)

LARO-BOT

다국어 명령을 로봇 행동으로 변환하고
실행 결과에 따라 다시 시도하거나 중단하는
ROS 2 로봇 팔 시스템

ROS2(Jazzy)

C++ / Python

OpenCV

Gazebo

Ollama

Docker



팀 프로젝트

[프로젝트 코드 ↗](#)

VICPINKY CARRIER

6대의 이기종 로봇을 연결해 작업을 배정하고
로봇 간 통행을 조율하고 운영 상태를 관리하는
통합 관제 시스템

ROS2(Jazzy)

NestJS

React

Socket.IO

WebRTC

MongoDB

개인 프로젝트

LARO-BOT

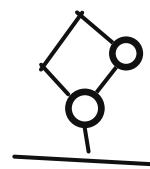
자연어 명령으로 물체를 집고 옮기는 로봇팔



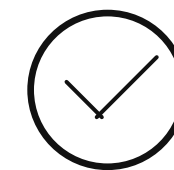
실물 로봇 개발 환경



자연어 명령

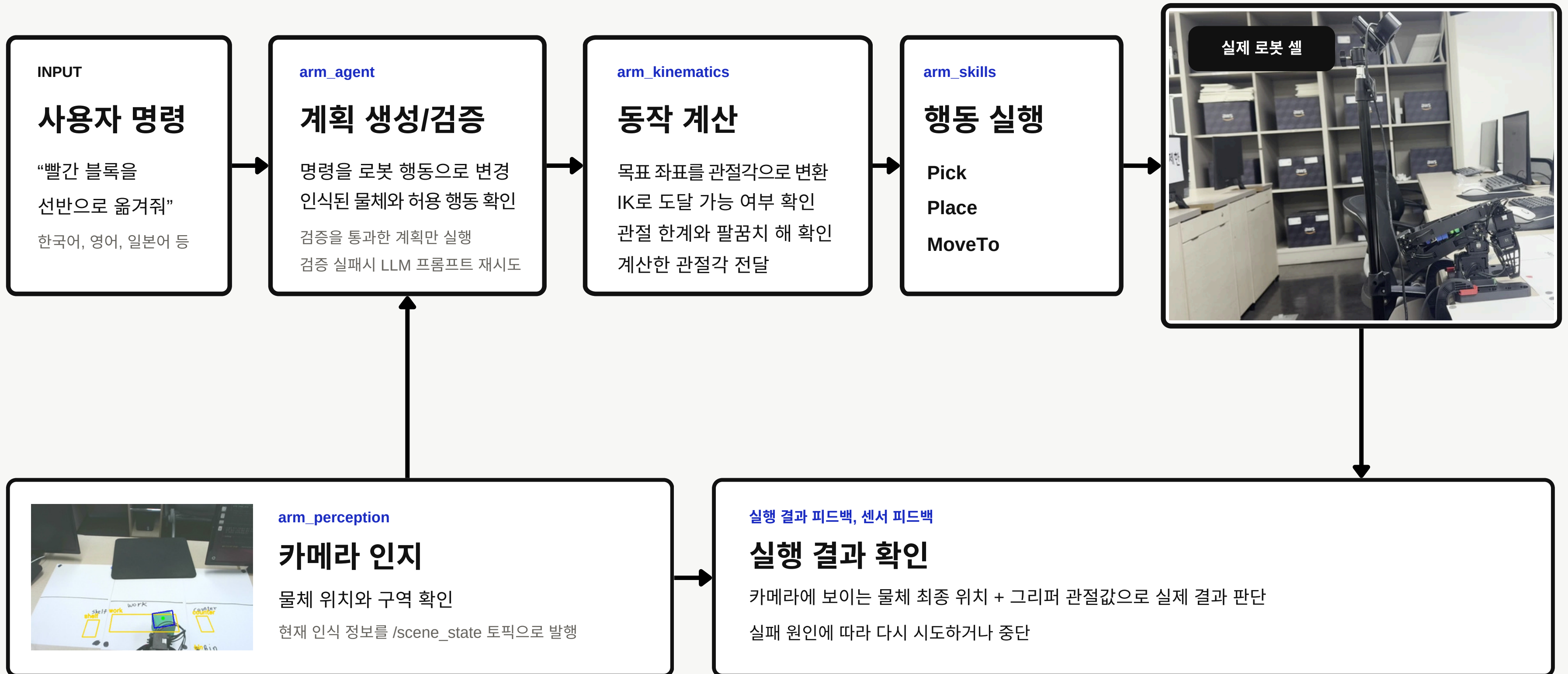


로봇팔 동작



실패 복구

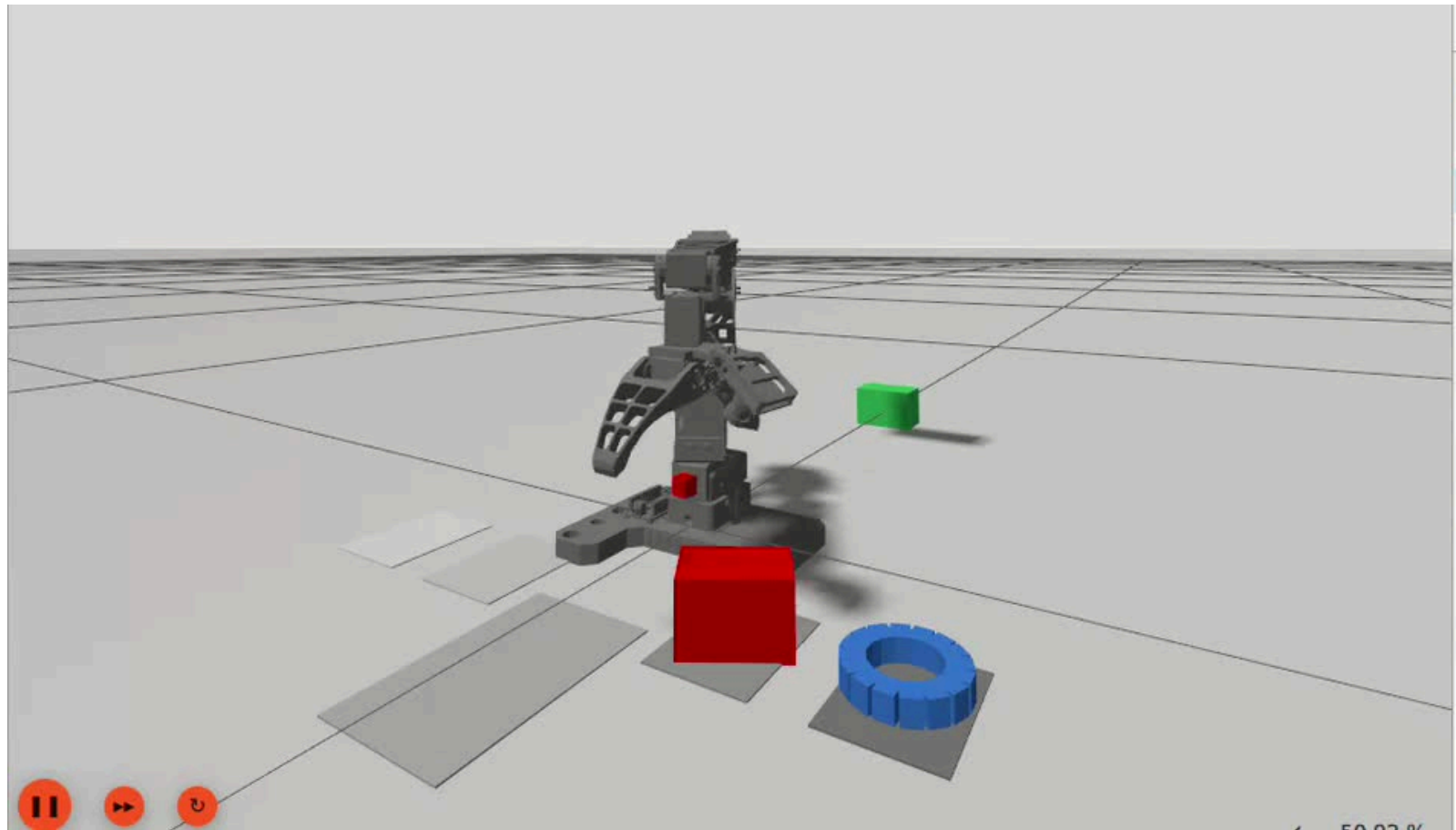
자연어 명령에서 실제 로봇 행동까지



전체 작업 흐름을 Gazebo에서 확인하고 실물로 연결

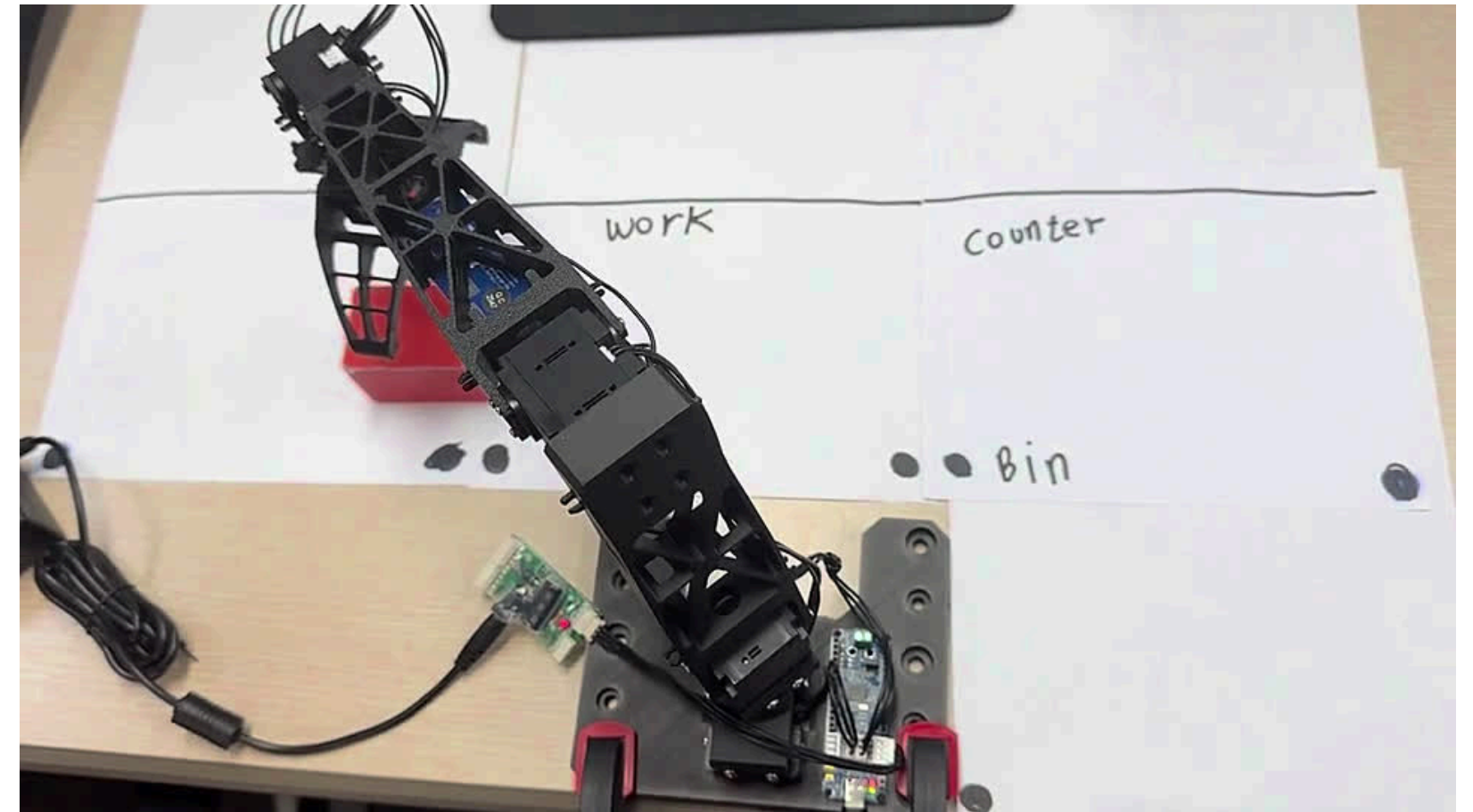
작업 예시 “빨간 블록을 카운터로 옮겨줘”

Gazebo : 전체 작업 흐름 확인



명령 → 집기 → 이동·놓기 → 결과 확인

실물 : Pick, Place 실행



실제 물체의 동작 확인

다국어 명령을 검증된 행동 계획으로 변환

사용자 명령 예시

“カウンターの赤いブロックを棚に移して”

카운터의 빨간 블록을 선반으로 옮겨줘

실행 전 계획 검증

- JSON 형식과 필수 인자 존재 여부 확인
- 카메라에 인식된 물체와 등록된 목적지 확인
- 허용된 행동 중 하나인지 확인

구조화된 계획

pick(red_block)

place(red_block, shelf)

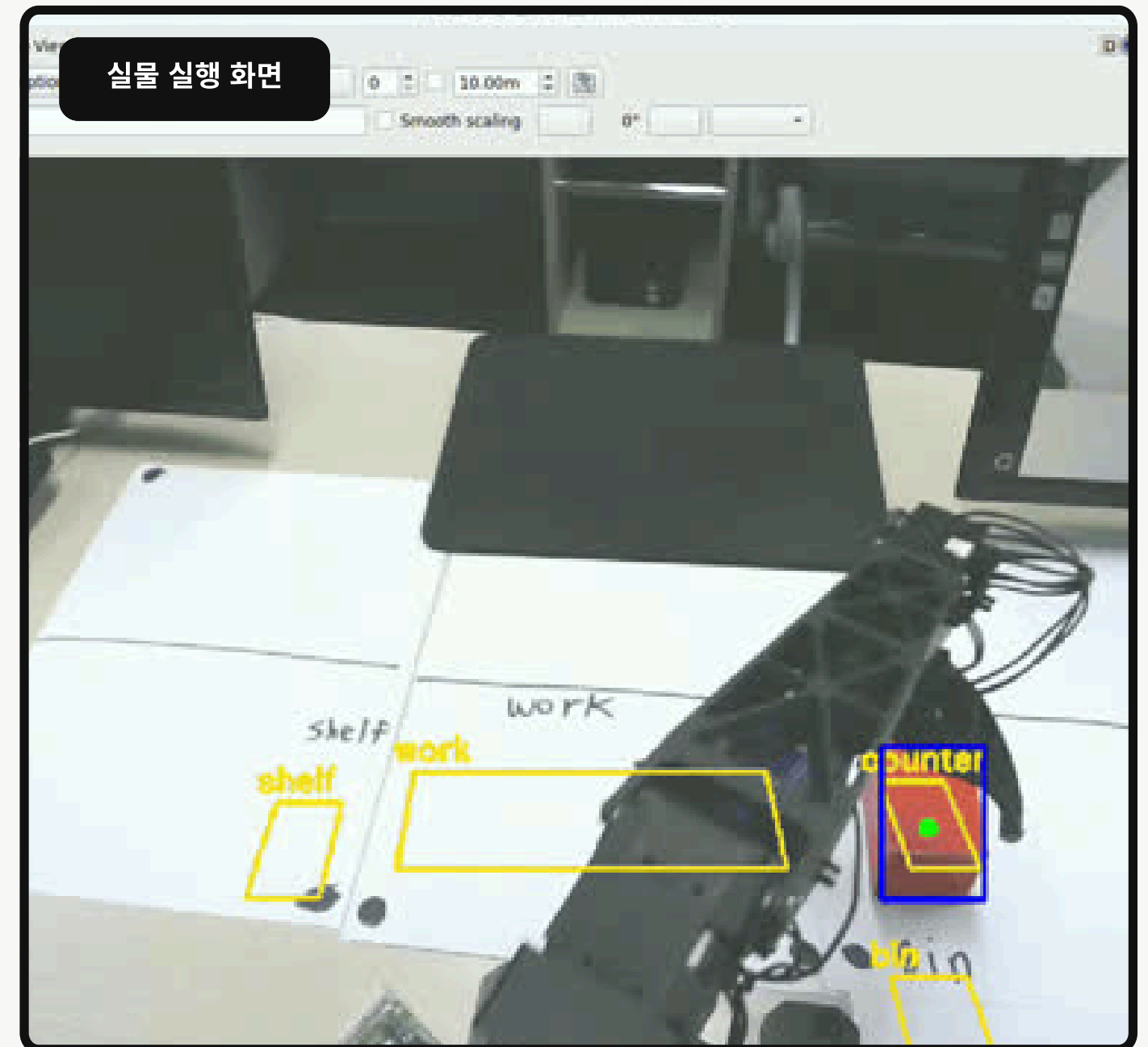
로봇 행동 최대 2단계

Gemma4 26B 계획 생성 테스트 결과

220 / 242 (90.9%)

10개 언어 명령 평가 결과

59 / 60 (98.3%)



카메라 인식 결과를 로봇 중심의 좌표점으로 공유

인지 과정

- 01 **HSV 색 범위로 물체 구분**
물체별 색상 범위 사용
- 02 **윤곽선에서 중심·방향 추출**
중심점과 긴 축 계산
- 03 **픽셀 좌표를 로봇 기준 좌표로 변경**
로봇의 위치를 0,0으로 하는 좌표로 변환
- 04 **/scene_state로 정보 공유**
물체 ID, 위치, 방향



목표 좌표를 실제 로봇 움직임으로 연결

관절값 계산 및 경로 실행

01 목표 위치와 방향 입력

카메라 인지 결과로 받은 좌표와 물체의 방향

02 관절 목표값 후보 계산

위, 아래 자세의 관절값 계산하여 후보값 도출

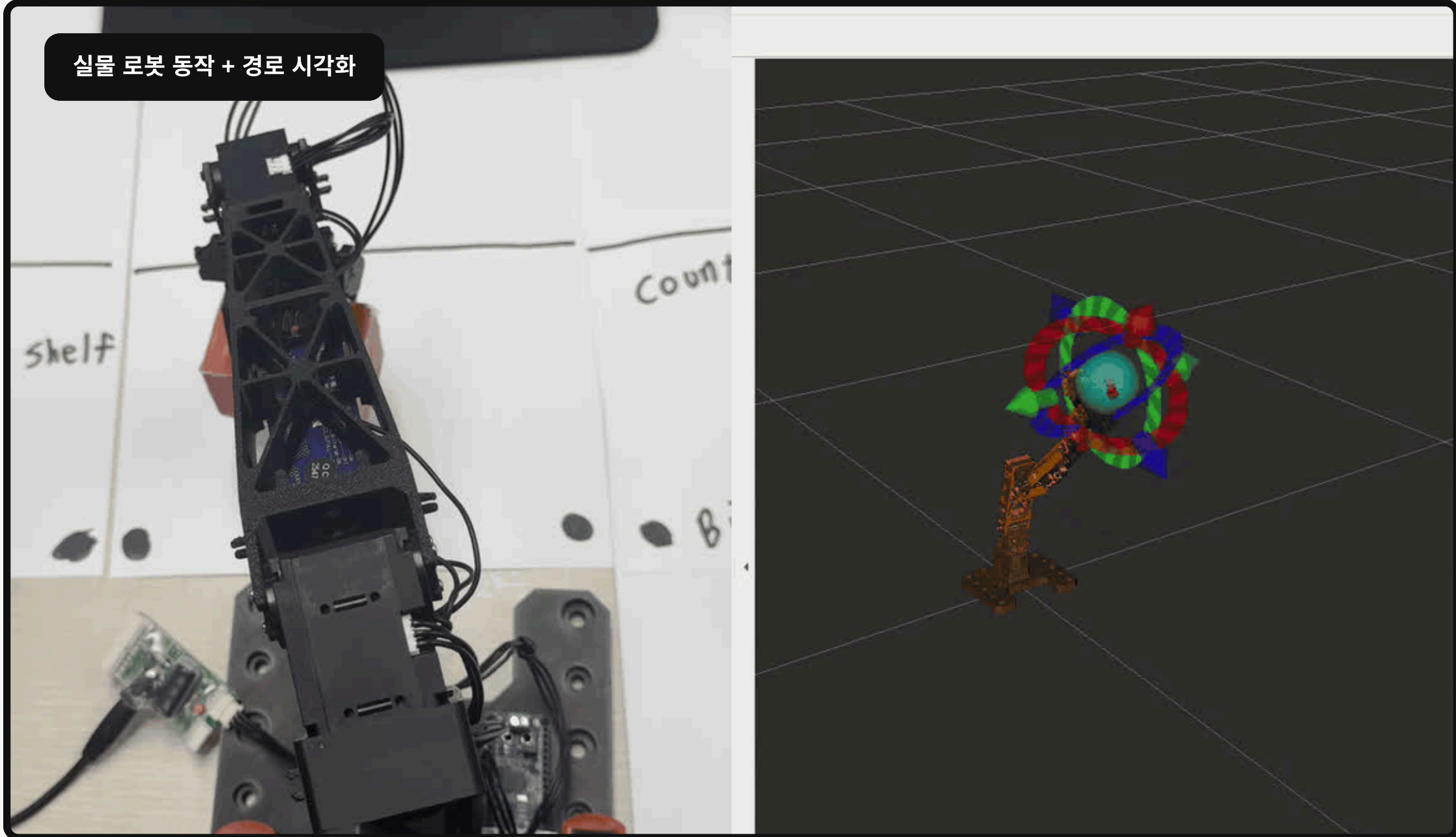
03 실행 가능한 자세 선택

관절 한계 확인 후 현재 자세와 가까운 후보값 선택

04 MoveIt 2로 경로 계획 및 실행

선택한 관절값을 컨트롤러로 전달

실물 로봇 동작 + 경로 시각화



로봇이 수행 가능한 세 가지 행동

Pick.action

Pick

물체를 집어 들어 올리기

물체 ID / 물체별 다른 접근 높이

이동 → 하강 → 접근 → 집기 → 상승

Place.action

Place

지정한 구역에 내려놓기

집은 물체 / 목적 구역

이동 → 낙하 확인 → 하강 → 놓기 → 후퇴

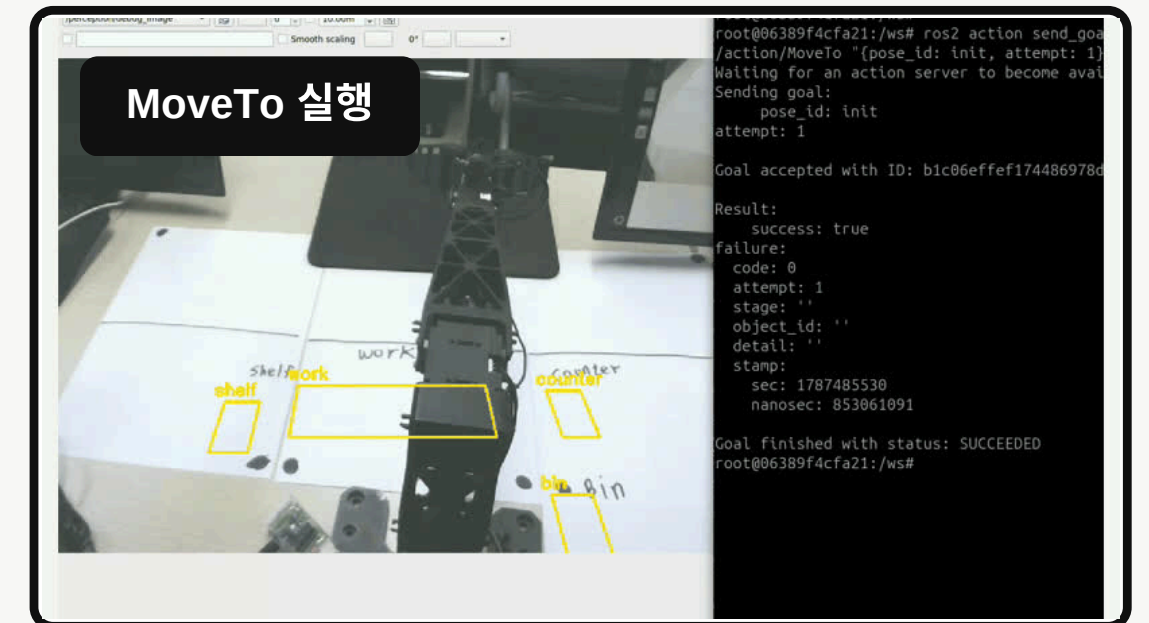
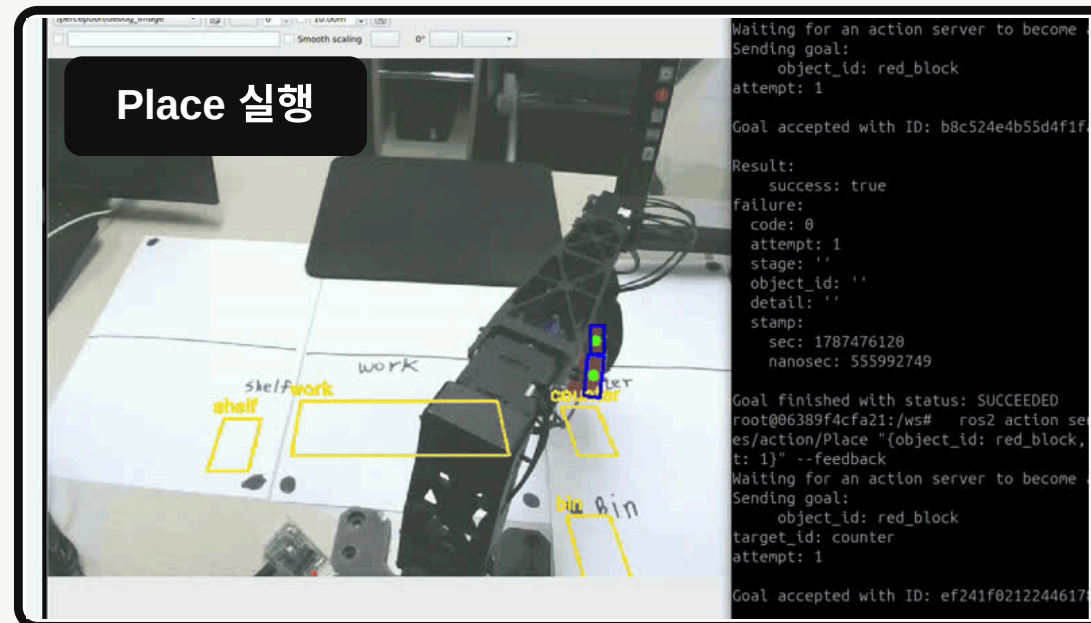
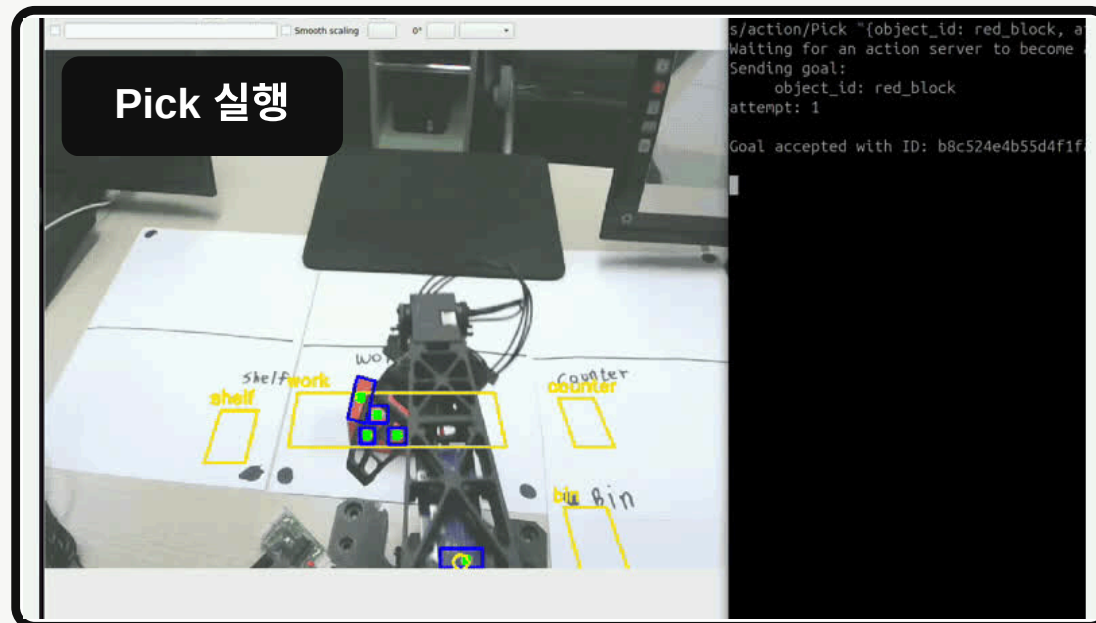
MoveTo.action

MoveTo

지정된 자세로 이동

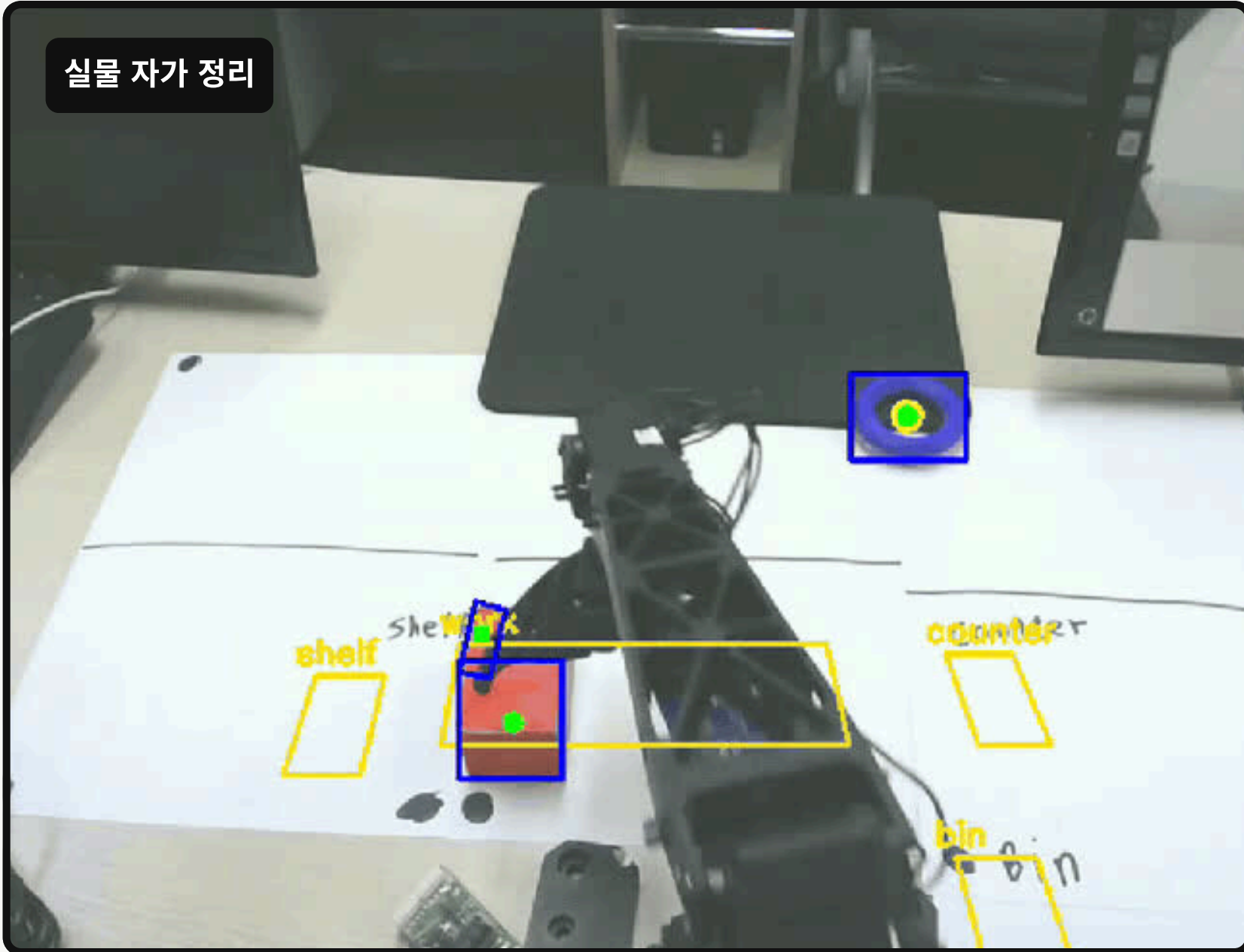
사전 정의된 자세

현재 자세 → 지정된 자세



작업 구역에 남긴 물건을 스스로 정리

실물 자가 정리



대기 상태에서 2초 이상 관찰

work 구역에 머무는 물체를 정리 대상으로 선택

물체별 정리 목적지 지정

red_block → shelf / green_block → bin

기존 행동 시퀀스 재사용

Pick → Place → MoveTo(home)

동작 후 최종 배치 확인

마지막 인식 결과로 배치 여부 확인

실패 원인에 맞춰 다시 시도하거나 중단

오류별 기본 복구 원칙

OBJECT_NOT_FOUND

물체가 보이지 않음

재탐색

1. 후퇴
2. 물체 위치 확인

GRASP_FAILED

빈손으로 닫힘

재파지

1. 후퇴
2. 물체 위치 확인
3. Pick

GRIPPER_EMPTY

운반 중 물체 낙하

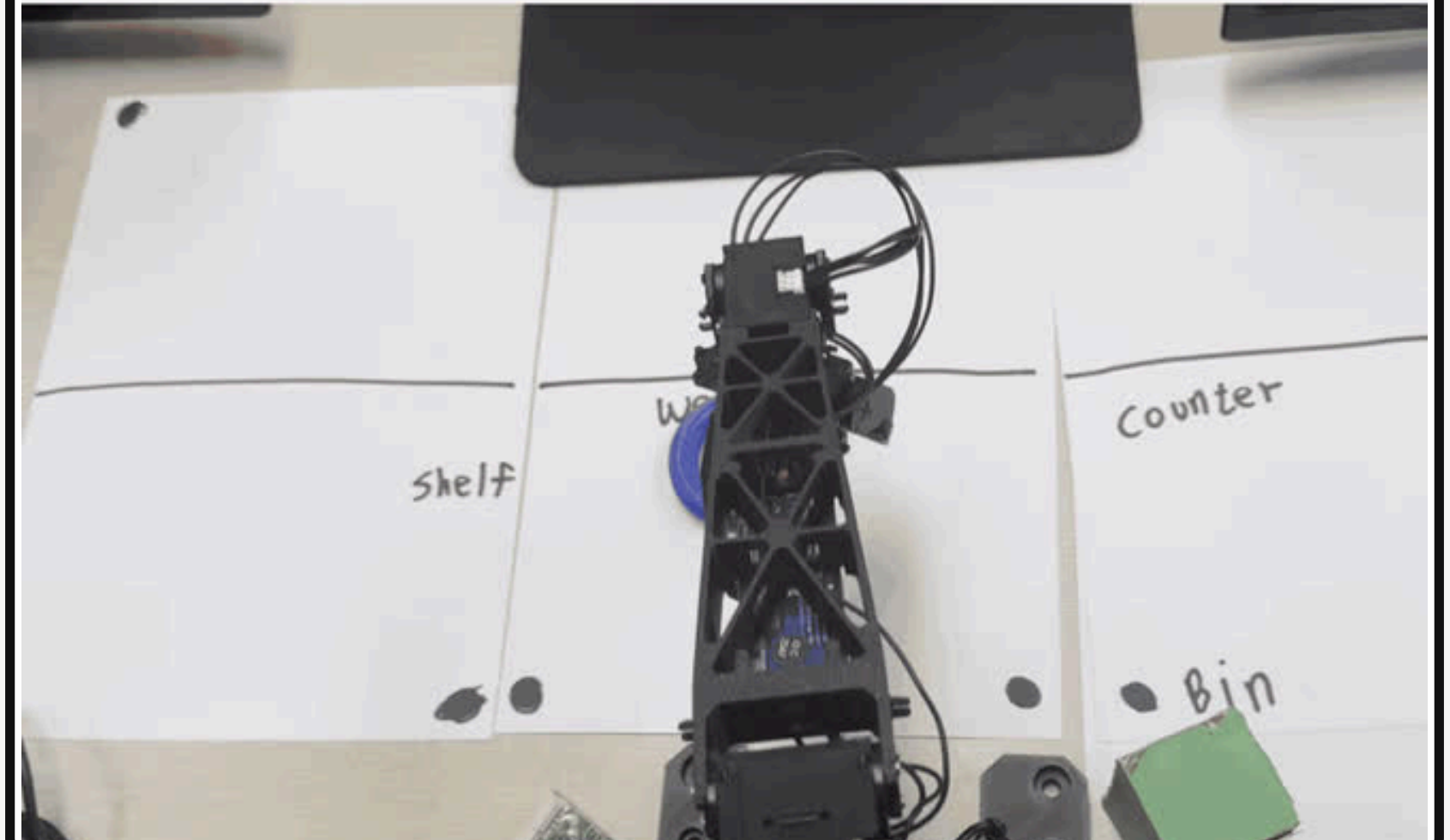
UNDEFINED_POSE UNREACHABLE

미등록 자세 / 도달 불가

작업 중단

복구 시도 없이 중단

GRASP_FAILED 복구



복구 정책

한 명령당 복구 최대 1회, 각 단계별 시도 횟수 제한.
중단 명령 혹은 복구 횟수 소진 시 작업 중단

수행 불가능한 명령 차단

01 중복 실행 차단

작업 중 들어온 추가 명령

기존 작업 실행 중

busy = true

새 Goal 수신

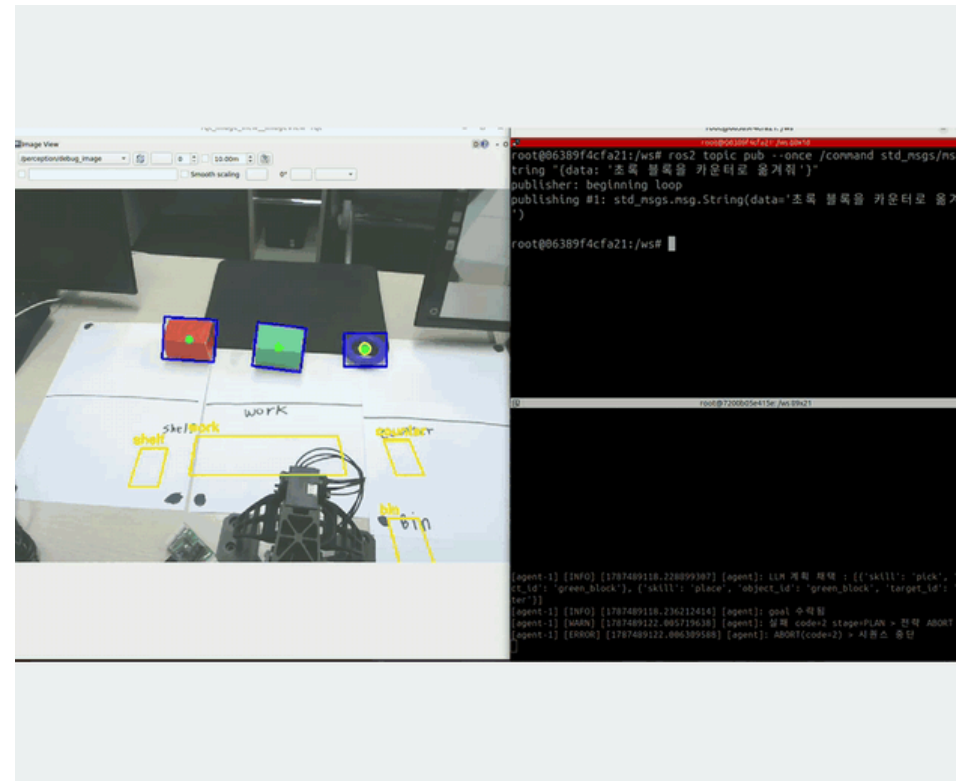
다른 작업을 수행하고 있는 지 확인

추가 요청 거부

기존 작업은 유지, 추가 요청만 거부

02 도달할 수 없는 목표 차단

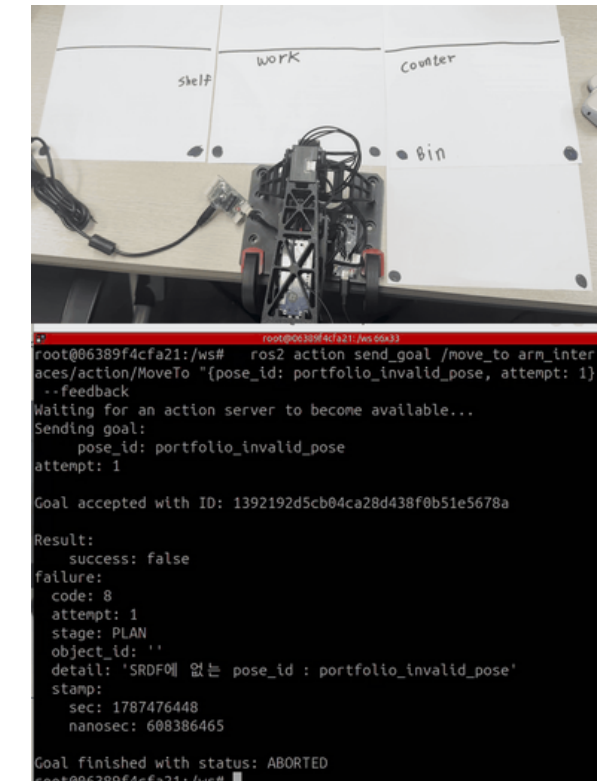
로봇팔이 닿을 수 없는 위치 요청



UNREACHABLE

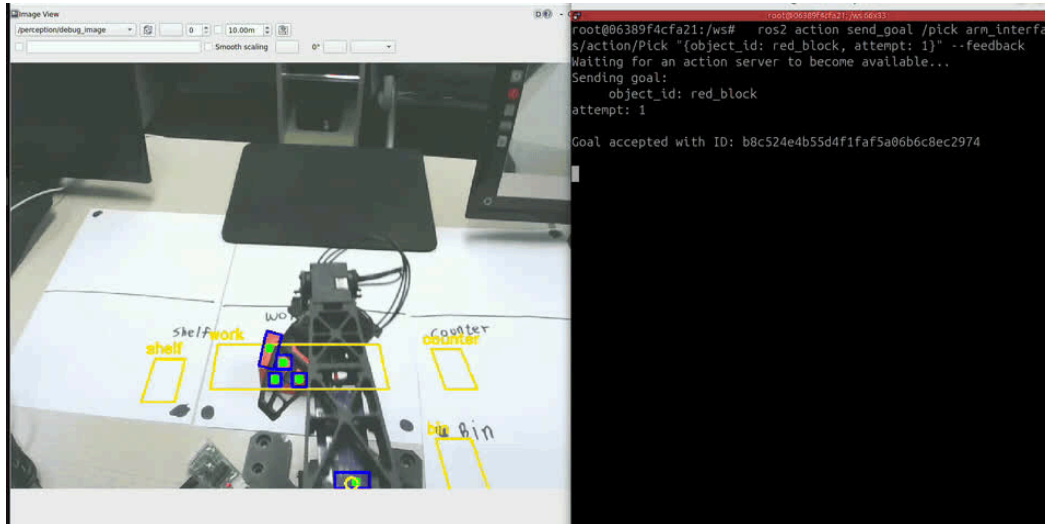
03 잘못된 자세 요청 거부

등록되지 않은 자세 이름 요청



UNDEFINED_POSE

개발 과정에서 발견한 문제 개선



실물 좌표 오차

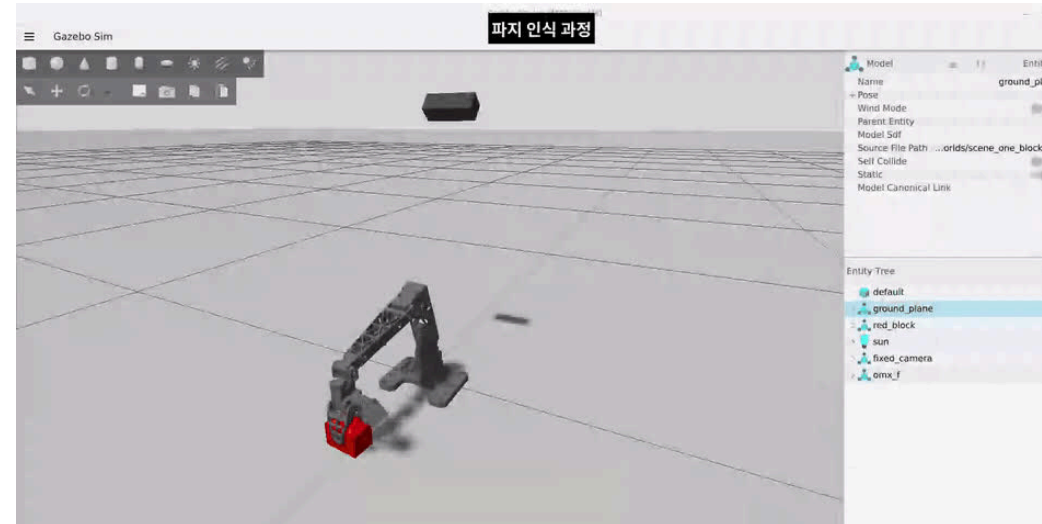
실측으로 좌표 오차 원인 분리

시뮬레이션 좌표 설정을 실물에 적용하자

추정 좌표와 실제 위치에 24~51mm 차이 발생

변경 팔의 도달 위치와 카메라 좌표를 분리 측정

결과 팔 도달 오차 1~6mm로 확인, 카메라 오차와 구분



성공 오판

그리퍼 관절값으로 직접 판정

컨트롤러 완료만으로 판단하자

빈손도 성공으로 처리되는 문제 발생

변경 그리퍼가 멈춘 뒤 관절 위치 구간을 직접 비교

결과 빈 손 / 블록 / 링 파지 상태 구분



급격한 반전

IK 해의 연속성을 유지

이동 중 팔꿈치 해가 바뀌면서

팔이 크게 반전하고 물체가 낙하

변경 현재 자세에서 가까운 IK 해를 계속 선택 + 파지 중 고정

결과 파지 중 팔꿈치의 급격한 변화 금지

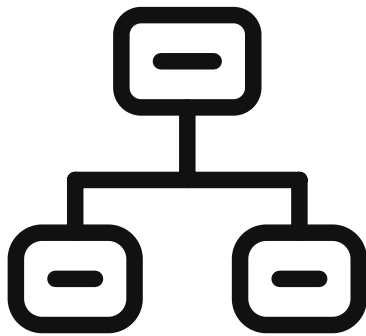
팀 프로젝트

VicPinky Carrier

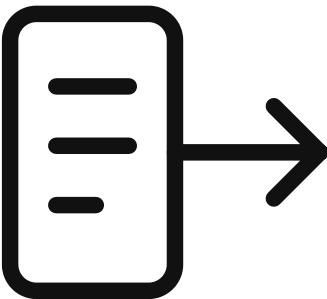
이기종 로봇 6대를 연결하는 통합 관제 시스템
7인팀, FMS 설계 담당



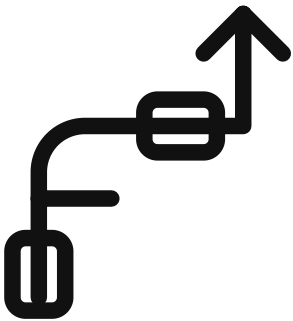
FMS 작업 진행 화면



다중 로봇 관제



작업 배정



교통 관리

조난자 좌표를 구호 작업의 목적지로 연결

01 전체 작업 일람 위치 표시

정찰 로봇 / RViz 표시

첫번째 요구조자 발견



01 조난자 좌표 수신

`/victim/confirmed`

정찰 로봇이 보낸 map 좌표를 수신

02 임시 노드·경로 연결

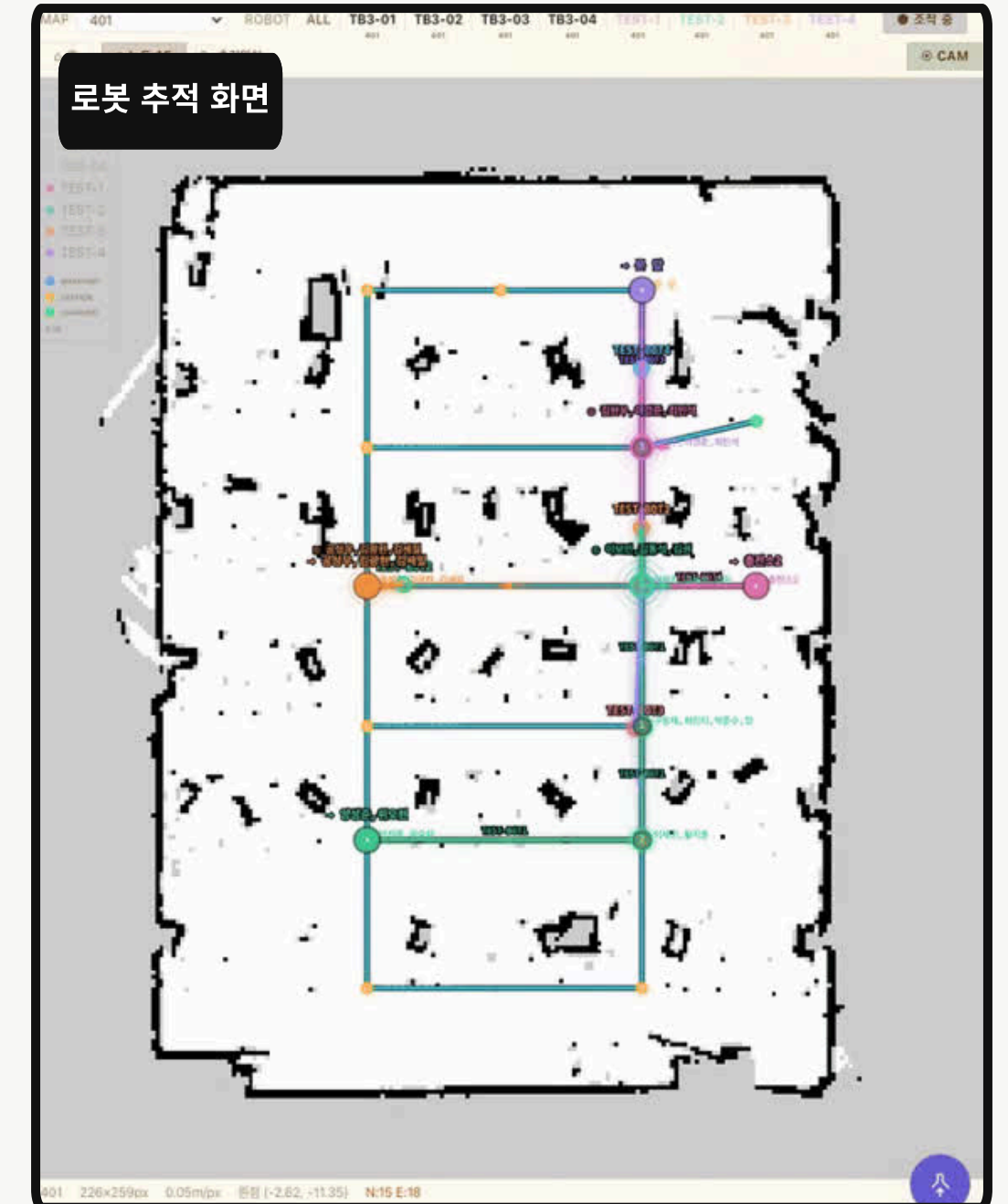
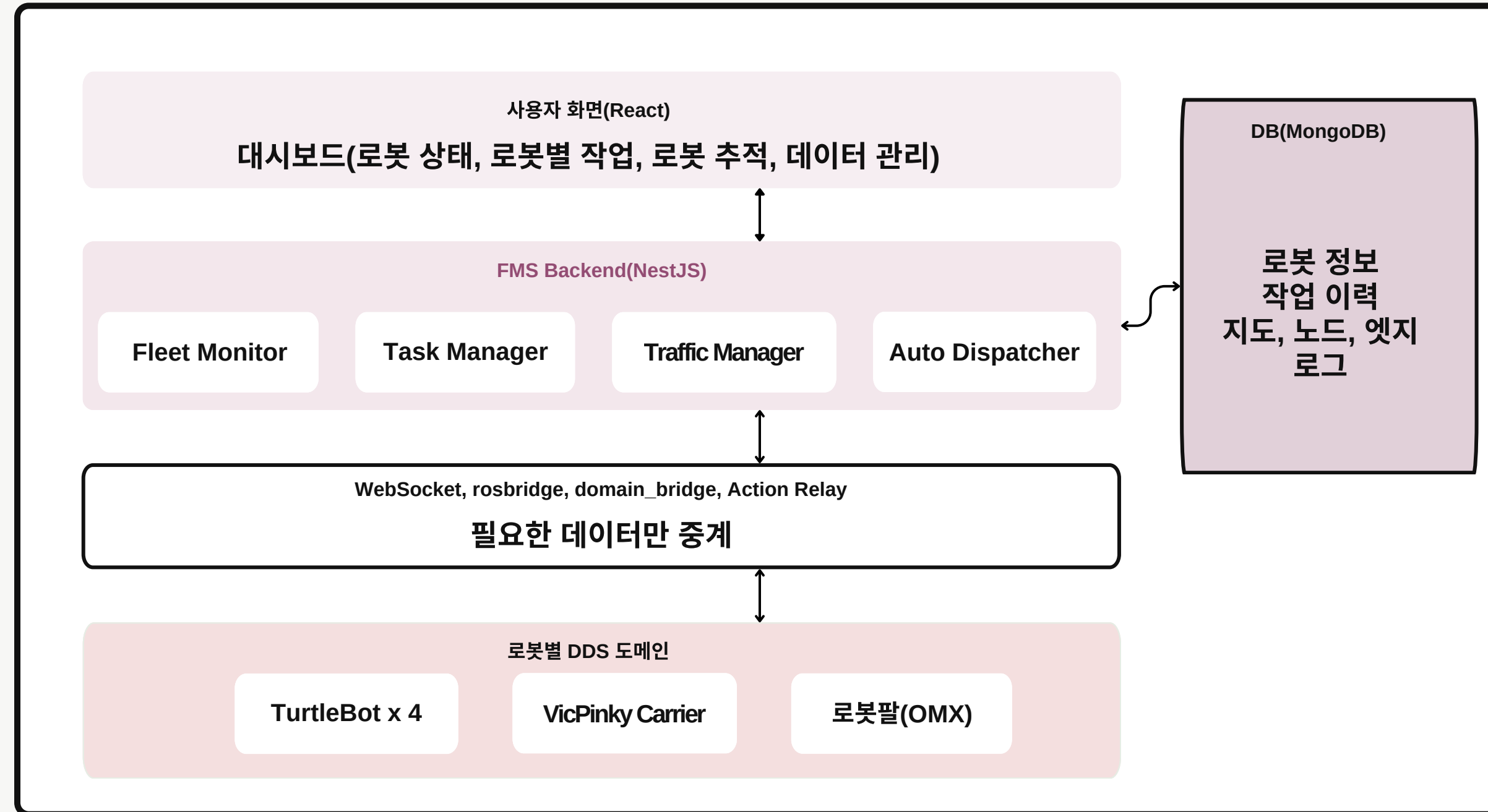
좌표를 VICTIM 노드로 등록

기존과 가장 가까운 노드와 엣지로 연결

03 구호 작업 생성

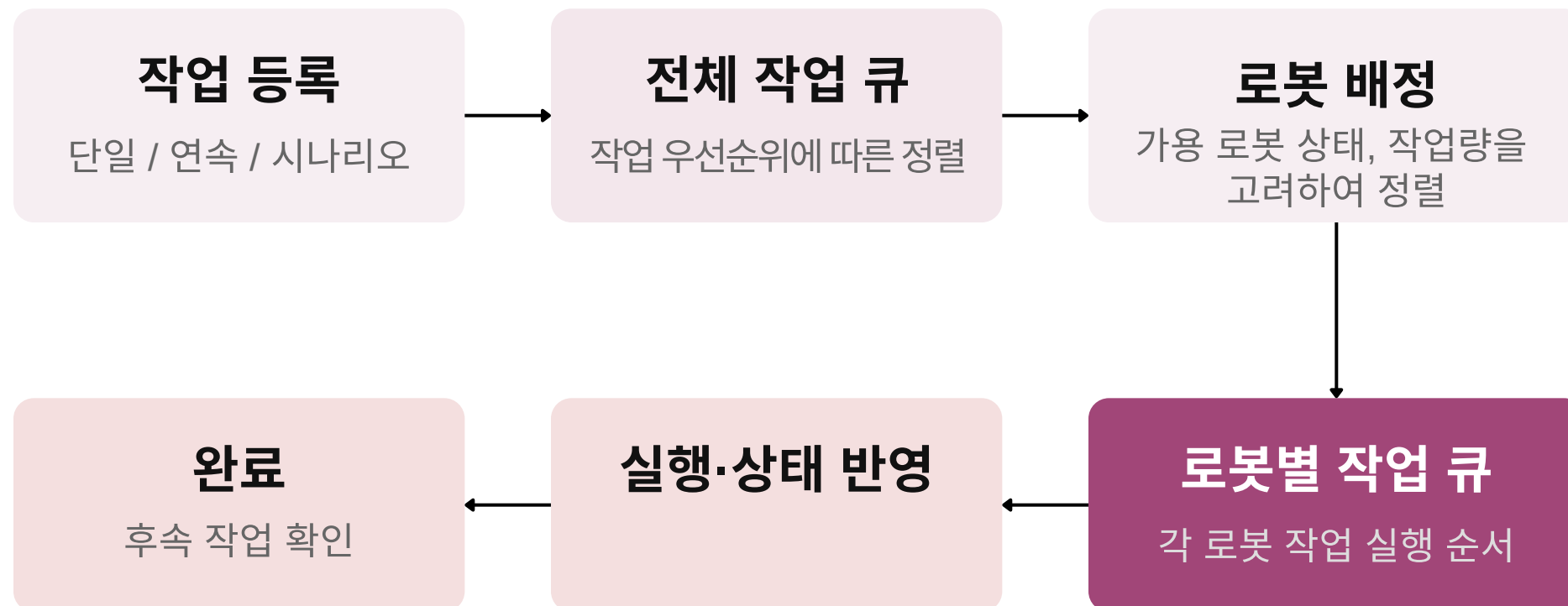
목적지 선택 → 구호 로봇 배정

여러 이기종 로봇을 계층과 도메인을 나눠 하나의 시스템으로 통합



작업 우선순위와 로봇별 실행 순서를 분리

작업 배정과 정상 실행



작업 진행 화면

연속 작업 진행



현재 수행 작업



관제 기능 테스트 · 로봇 상태 모사

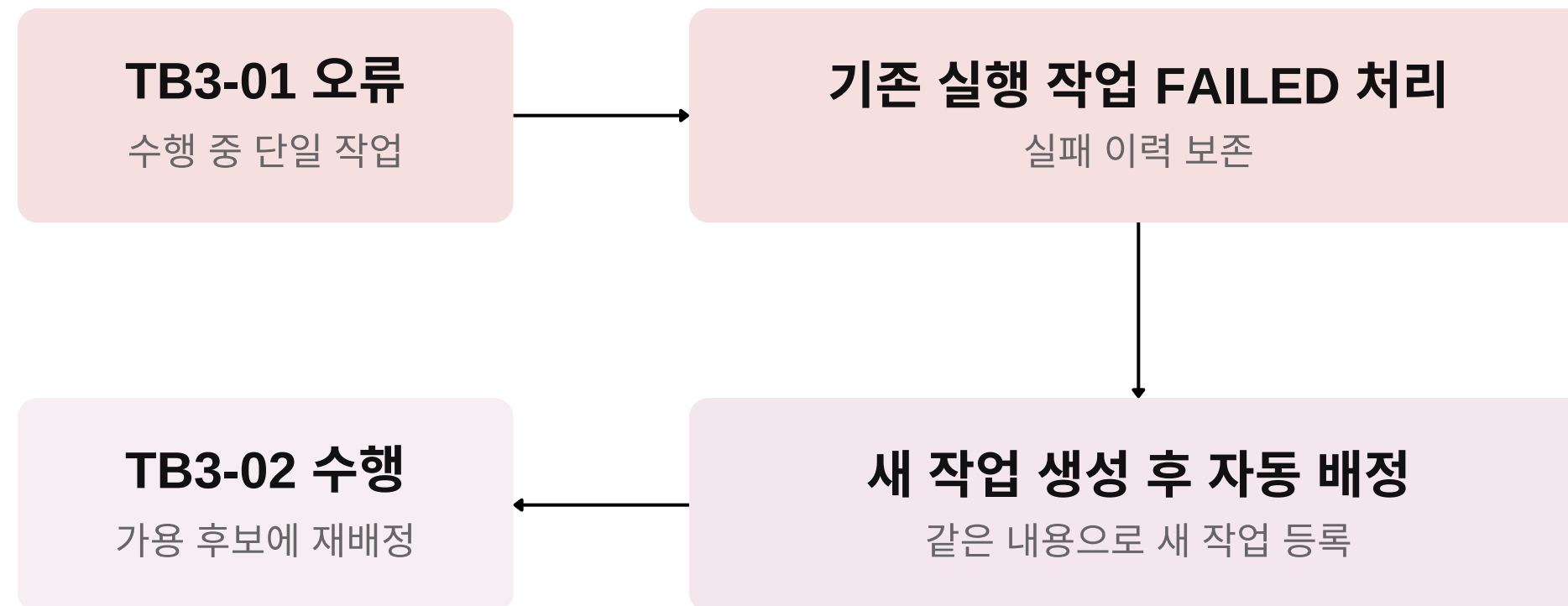
완료된 단계와 현재 수행 작업을
같은 화면에서 확인

후보 로봇 비교 **배정된 작업 수 적은 순 → 배터리 잔량 높은 순 → 목적지에 가까운 순**

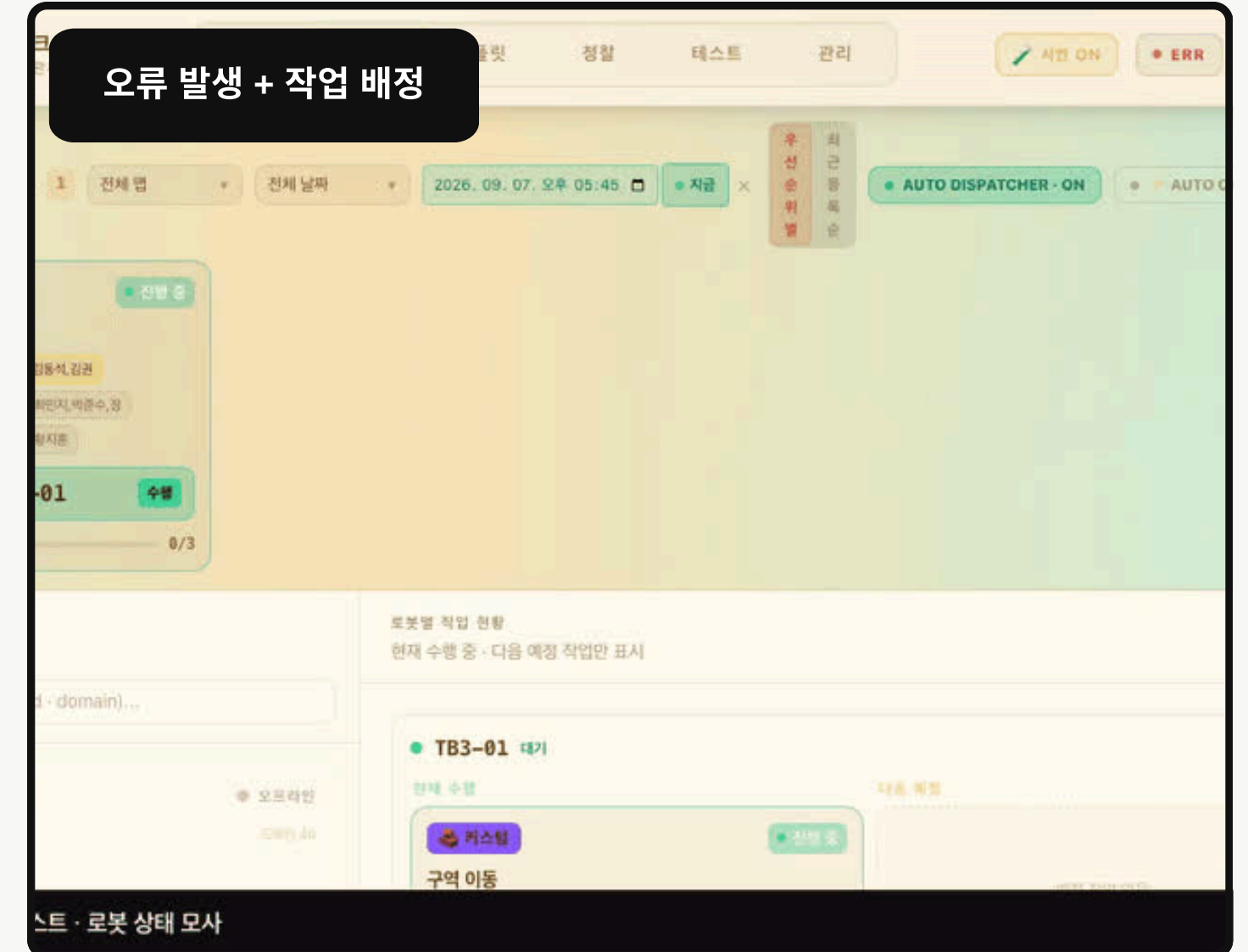
로봇 상태 오프라인, 오류 발생(ERROR), 일시정지(PAUSED) 제외

실패 이력을 남기고 단일 작업을 재배포

수행 중인 단일 작업의 오류 처리



자동 배정 실행 시 작업 순서

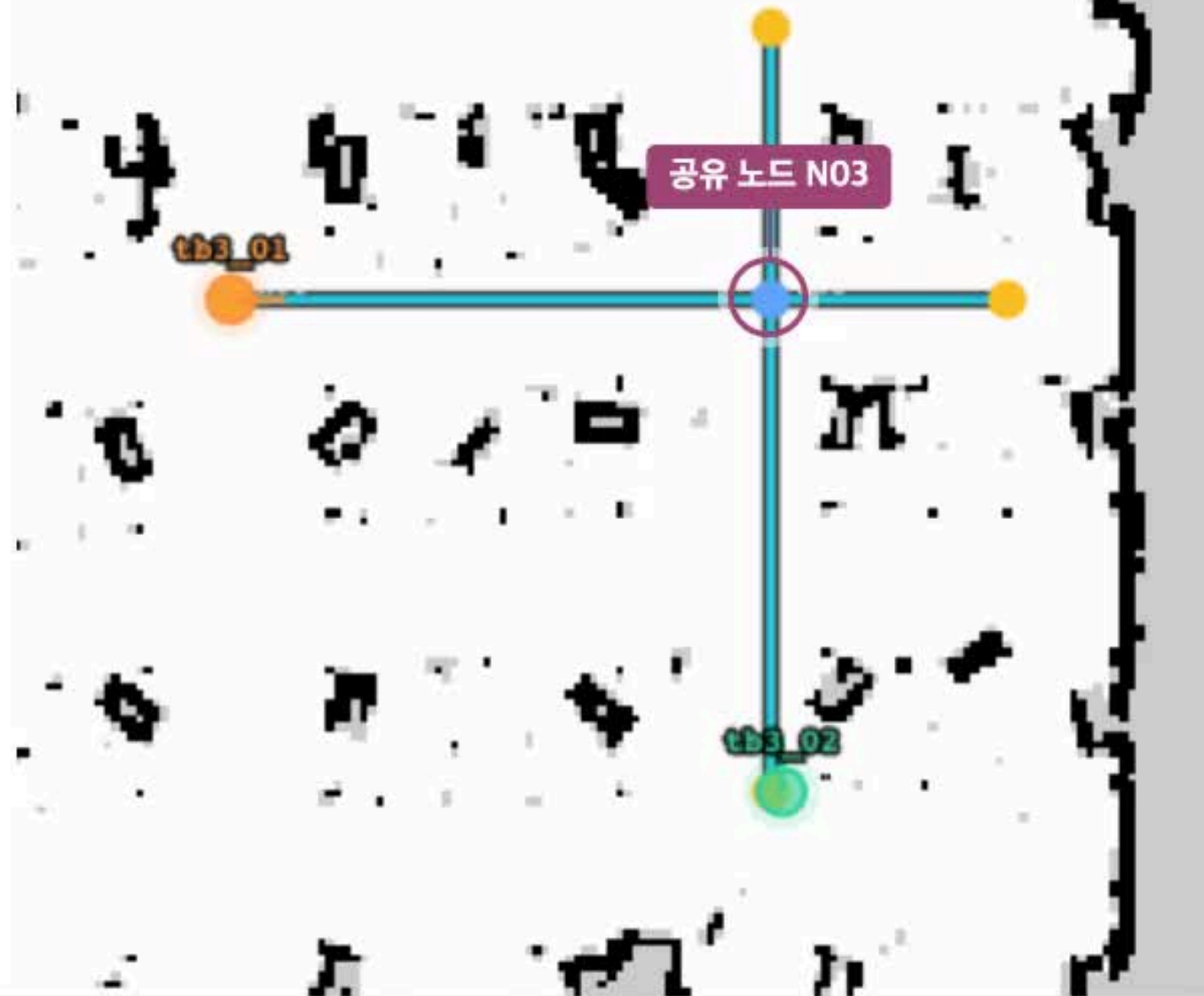


연속 / 시나리오

한 로봇에게 여러 작업을 동시에 할당했을 경우
완료 이력은 보존하고, 남은 작업은 실패로 정리

로봇 간 충돌 방지

충돌 방지



경로 우선권 결정 순서

작업 유형 우선순위

착수 시각

로봇 ID

착수 시각이 없으면 생성 시각을 사용합니다.

01 두 로봇의 다음 목표가 같은 노드로 겹침

다음 노드의 점유 상태와 다른 로봇의 목표를 비교

02 구호는 진행, 이동은 양보 대기

구호(PROCESS) > 이동(MOVE) > 충전(CHARGE)

03 조건 해소 후 남은 경로로 재출발

tb3_01은 N04에서 완료

tb3_02는 N03 → N05로 이동을 이어감

경로 생성 : Dijkstra

잠긴 중간 노드를 제외해 경로 선택

경로 주행 시 : 다음 노드 조회

경로 우선권에 따라 대기 혹은 재출발 결정

로봇 배터리 부족 시 자동 충전

자동 충전

플릿 명령 할당
동작, 문제, 설정

제어 | 태스크 | 플릿 | 정찰 | 테스트 | 관리

시연 로봇 ON | CONN | NEST

전체 날짜: 2026. 09. 09. 오전 09:56 | 지금 | 우선순위별 | 최근 등록순 | AUTO DISPATCHER: OFF | AUTO CHARGE: ON | + 태스크 추가

연속 tb3_01 1/2

1 이동 → N1 진행 중

2 이동 → N2 대기 중

MAP 401 | ROBOT ALL | TB3-01 | TB3-02 | TB3-03 | TB3-04 | tb3_01 | tb3_02 | tb3_03 | tb3_04 | 조작 중 | 홈 | 노드 5 | 초기위치 | CAM

tb3_01 · API 관측값

15%

WORKING

작업 큐 2건 · 현재 노드 INIT

02 배터리 15%여도 배정된 작업부터 완료
현재 실행 + 대기 작업이 0이 될 때까지 충전 이동 보류

FMS 상태 시뮬레이션

자동 충전 순서

- 배터리 20% 미만**
충전 필요 상태로 전환
- 배정된 작업을 모두 완료**
현재 실행 + 대기 작업이 모두 0건이면 진행
- 빈 충전소로 이동**
빈 충전소가 없으면 지정된 장소에서 대기
- 80% 이상이면 지정된 장소로 복귀**
충전소를 비우고 지정된 장소에서 대기

개발 과정에서 발견한 문제 개선

중계 목록



통신 병목

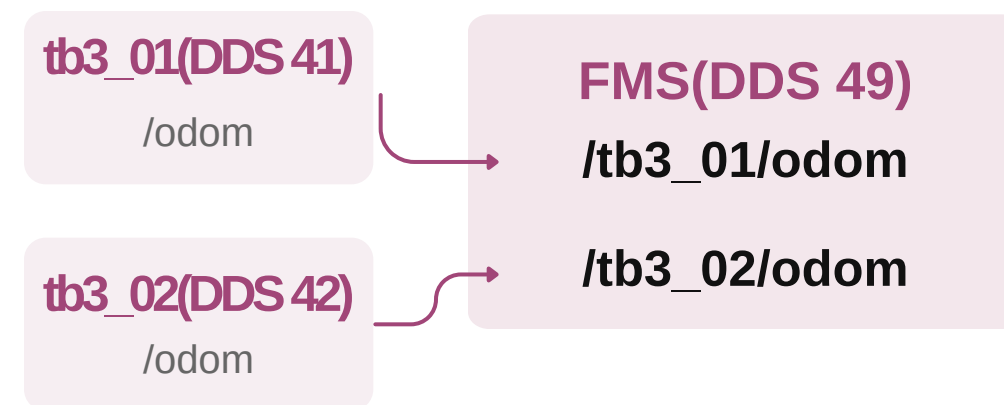
불필요한 중계 트래픽 축소

고빈도 TF까지 중계하며
상태 수신이 불규칙해짐

변경 로봇별 중계 목록을 조정하고
관제용 위치 상태 토픽을 사용

결과 필요한 관제 데이터 중심으로
중계 범위를 정리

로봇별 통신 분리



로봇 간 토픽 구분

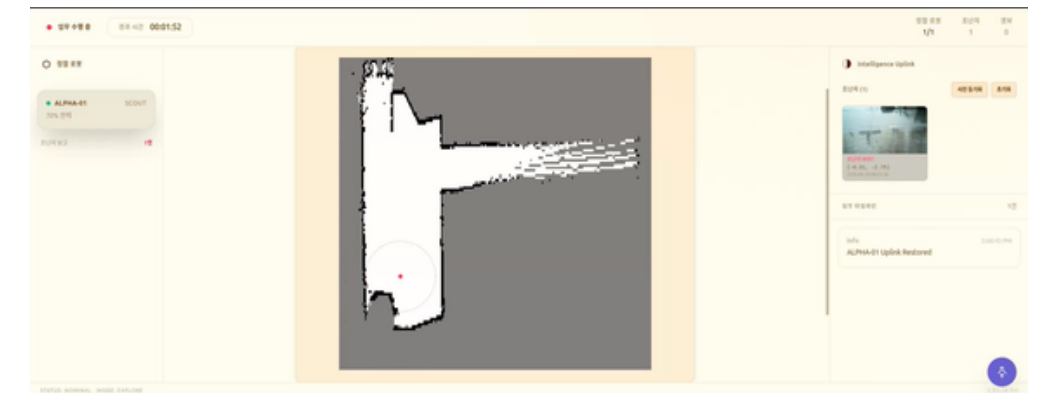
로봇별 DDS 도메인 분리

여러 로봇이 같은 토픽명을 사용해
상태와 명령을 구분할 필요

변경 로봇별 도메인을 나누고
허브에서 로봇 ID로 구분

결과 로봇 내부 토픽명은 유지하고
관제에서 상태·명령을 분리

다중 로봇 주행 관리



노드 기반 관제

좌표 전달에서 노드 기반 관제로 확장

여러 로봇의 이동을 관리하려면
위치와 경로 진행을 비교할 공통 기준이 필요

변경 경로를 노드, 엣지로 표현하고
위치 정보로 상태와 통행 충돌을 판단

결과 FMS는 경유 순서와 통행을 조율하고
Nav2는 노드 사이의 주행을 담당

THANK YOU



박준수

pjsu94@naver.com

github.com/kdm111